Gift of the Givers · Relief Management System

DELIVERABLE 11

Azure Deployment and GitHub Sync Guide

Step-by-step instructions for the team member who sets up Azure, connects GitHub to Azure DevOps and turns on continuous deployment.

DOCUMENT	VERSION	DATE	STATUS
D-11	1.0	2026-11-02	Ready to follow — no step has been carried out in Azure yet

CONTENTS

1. What you will end up with
2. Before you start: access tokens
3. Create the Azure resources
4. Prepare Azure DevOps
5. Sync GitHub to Azure DevOps
6. Create the pipeline
7. What each deployment stage does
8. After the first deployment
9. Rollback and restore
10. Troubleshooting
11. Hand-over checklist
12. Costs and clean-up

1. What you will end up with

When you finish this guide:

- Every push to the **GitHub** repository is copied automatically to **Azure Repos** (Azure DevOps).
- Each copy starts the **Azure Pipeline** (`azure-pipelines.yml`). It builds the solution, runs the unit, integration and Selenium tests, publishes test and coverage reports, then on `main` it:
  1. applies database migrations to **Azure SQL**,
  2. deploys the web app to **App Service** (through a staging slot if your plan has one),
  3. deploys the tax-certificate **Azure Function**, and
  4. runs read-only Selenium smoke tests against the live site.
- The live site at `https://gift-relief-managemen-gdbsf9bbena9epgw.southafricanorth-01.azurewebsites.net` shows the application instead of the Azure "waiting for your content" page.

The path of one change: developer pushes to GitHub → GitHub Action *Mirror to Azure DevOps* → Azure Repos → Azure Pipeline *Build and test* → (approval) → *DeployDatabase* → *DeployWebApp* and *DeployFunctions* → *LiveSmokeTests*.

Time needed: about 2 hours the first time. **You need:** a browser, Owner or Contributor access to the team's Azure subscription, Project Administrator access to the Azure DevOps organisation `ST10438928` , and Admin access to the GitHub repository `jBlack-MC/GiftOfTheGivers-HumanitarianAid` . Installing nothing locally is fine; the Azure CLI is optional.

> **Write things down as you go.** Section 11 is a checklist of every name, URL and secret you create. Keep it in a password manager, never in the repository, a chat or a document.

2. Before you start: access tokens

Personal access tokens (PATs) are passwords. Rules for this project:

- **A PAT that has been pasted into a chat, email or document is compromised.** Revoke it (Azure DevOps → your avatar → *Personal access tokens* → *Revoke*) and create a new one. A token was shared in this way during Part 3; when tested on 2026-11-02 it was already rejected by Azure DevOps (HTTP 401), but revoke it anyway if it still appears in the list.
- Create each PAT with the **smallest scope** and an expiry date at the end of the module.
- Store PATs only as **GitHub repository secrets** or **secret pipeline variables**.

You will create two PATs in section 5:

PAT	Scope	Stored as
Repository mirror	*Code: Read & write*	GitHub secret `AZURE_DEVOPS_PAT`
Helper package publishing (Part 2)	*Packaging: Read & write*	GitHub secret `AZURE_ARTIFACTS_PAT`

3. Create the Azure resources

Use the Azure portal (). Use **South Africa North** for every resource, because the existing web app is already there. The names below are the defaults in `pipelines/templates/variables.yml`. If a name is taken, choose another and write it down; you will change the file in section 7.

3.1 Resource group

1. Search **Resource groups** → **Create**.
2. Subscription: the team subscription. Name: `rg-giftofthegivers`. Region: *South Africa North*.
3. **Review + create** → **Create**.

> The existing web app may already be in a different resource group. Open it (search **App Services** → `gift-relief-management…`) and look at *Resource group* on the Overview page. If it is different, either use that group everywhere below or move the app (*Overview* → *Move* → *Move to another resource group*).

3.2 Azure SQL server and database

1. Search **SQL databases** → **Create**.
2. Resource group `rg-giftofthegivers`. Database name `GiftOfTheGiversDb`.
3. Server → **Create new**: name `gift-relief-sql`, location *South Africa North*, authentication **Use SQL authentication**. Admin login, for example `gotgadmin`, and a strong password. **Write both down.**
4. *Want to use SQL elastic pool?* **No**. *Workload environment*: **Development**.
5. **Compute + storage** → **Configure database**: choose **Basic (5 DTU, 2 GB)**, or the *free serverless offer* if the subscription shows it. Both fit the module's B1 limits.

6. *Backup storage redundancy*: **Locally-redundant** (cheapest).
7. **Networking** tab: connectivity **Public endpoint**. Turn on **Allow Azure services and resources to access this server** (the web app, function and pipeline need it). Optionally also *Add current client IP address* so you can query from your PC.
8. **Review + create** → **Create**. Wait about 5 minutes.
9. Open the database → **Settings** → **Connection strings** → **ADO.NET (SQL authentication)**. Copy it and replace `{your_password}` with the real password. This is `sqlConnectionString`. It looks like:

```
Server=tcp:gift-relief-sql.database.windows.net,1433;Initial Catalog=GiftOfTheGiversDb;Persist Security Info=False;User ID=gotgadm
```

You do **not** need to create any tables. The pipeline creates and updates the schema.

3.3 Web app (App Service)

The web app already exists: `gift-relief-management…` on **Windows** (it currently shows the Azure placeholder page).

1. Open it. On **Overview** note the **Name**, the **Resource group**, the **Default domain** (the long `…azurewebsites.net` address) and the **App Service plan** pricing tier.
2. **Settings** → **Configuration** → **General settings** (or *Environment variables* in the newer portal):
   - *Stack*: **.NET**, version **.NET 10 (LTS)**.
   - *Platform*: **64 Bit**. *HTTPS Only*: **On**. *Always on*: On if the tier allows it (B1 and above).
   - **Save**.
3. **Health check** (under *Monitoring*): enable it with path `/health`. App Service then restarts instances that stop answering.
4. **Deployment slots** (only if the plan is **Standard S1 or higher**): **Add slot** → name `staging`, clone settings from production. Copy the slot's URL; it is `webAppStagingUrl`. On Free or Basic plans, skip this step and leave `useStagingSlot: 'false'`.

The pipeline sets all application settings itself (connection string, `Database__MigrateOnStartup=false`, and so on). You do not need to type them in the portal.

3.4 Function app

1. Search **Function App** → **Create** → hosting plan **Consumption** (Windows).
2. Resource group `rg-giftofthegivers`. Name `gift-relief-functions` (must be unique; write down what you use).
3. *Runtime stack* **.NET**, *Version* **10 (LTS), isolated worker model**, *Region* South Africa North, *Operating system* **Windows**.
4. **Storage**: let it create a new storage account. **Monitoring**: Application Insights **Yes** (gives you logs for the report).
5. **Review + create** → **Create**.

You will copy the function key **after the first deployment** (section 8).

3.5 (Optional) Azure CLI equivalent

If you prefer the command line (`az login` first), this script creates the same resources. Replace the password placeholder, and pick unique names if needed.

```
RG=rg-giftofthegivers; LOC=southafricanorth
az group create -n $RG -l $LOC
az sql server create -g $RG -n gift-relief-sql -l $LOC -u gotgadmin -p '<strong-password>'
az sql server firewall-rule create -g $RG -s gift-relief-sql -n AllowAzure --start-ip-address 0.0.0.0 --end-ip-address 0.0.0.0
az sql db create -g $RG -s gift-relief-sql -n GiftOfTheGiversDb --service-objective Basic --backup-storage-redundancy Local
az storage account create -g $RG -n giftrelieffuncstore -l $LOC --sku Standard_LRS
az functionapp create -g $RG -n gift-relief-functions --consumption-plan-location $LOC \
  --runtime dotnet-isolated --runtime-version 10 --functions-version 4 --os-type Windows \
  --storage-account giftrelieffuncstore
```

4. Prepare Azure DevOps

Open and the team project (the one that holds the `giftgivers-helpers` feed from Part 2).

4.1 Azure Repos repository

1. **Repos → Files**. If the project has no repository named `GiftOfTheGivers-HumanitarianAid`, use the repository drop-down → **New repository**: Git, that name, **no** README.
2. Click **Clone** and copy the HTTPS URL, e.g. `https://ST10438928@dev.azure.com/ST10438928/<Project>/_git/GiftOfTheGivers-HumanitarianAid`. Remove the `ST10438928@` part. This is `AZURE_DEVOPS_REPO_URL`.

4.2 Service connection to Azure

1. **Project settings → Service connections → New service connection → Azure Resource Manager**.
2. *Identity type*: **App registration (automatic)**. *Credential*: **Workload identity federation**.
3. *Scope level*: **Subscription**; pick the subscription and resource group `rg-giftofthegivers`.
4. *Service connection name*: `giftgivers-azure-connection`. Tick **Grant access permission to all pipelines** (or authorise it on the first run).
5. **Save**.

If your student account cannot create app registrations, ask the subscription owner to create the connection, or use *Identity type: Managed identity* if offered.

4.3 Environment with an approval

1. **Pipelines → Environments → New environment** → name `giftgivers-production`, resource **None** → **Create**.
2. ⋮ → **Approvals and checks → Approvals** → add the deployment lead (and a second member if you want four-eyes). Every production deployment now waits for that approval. This is your release gate.

4.4 Variable group with the secrets

1. **Pipelines → Library → + Variable group**. Name: `giftgivers-azure` (exact spelling).
2. Add these variables. Click the **lock** icon for every row marked secret:

Variable	Value	Secret
`sqlAdminUser`	the SQL admin login (e.g. `gotgadmin`)	yes
`sqlAdminPassword`	the SQL admin password	yes
`sqlConnectionString`	the full ADO.NET connection string from 3.2	yes
`functionKey`	`pending` for now; replaced in section 8	yes

3. **Pipeline permissions** → **+** → allow the pipeline (after it exists in section 6), or answer *Permit* on the first run.
4. **Save**.

5. Sync GitHub to Azure DevOps

The repository already contains `.github/workflows/mirror-to-azure-devops.yml`. On **every push to any branch** it runs `git push --mirror` to Azure Repos. GitHub stays the place where the team works, and Azure Repos is an exact copy that Azure Pipelines builds from.

5.1 Create the mirror PAT

1. Azure DevOps → avatar (top right) → **Personal access tokens** → **New Token**.
2. Name `github-mirror`. Organisation `ST10438928`. Expiry: end of the module.
3. Scopes → **Custom defined** → **Code: Read & write** only. **Create**, then copy the token immediately (it is shown once).

5.2 Add the GitHub secrets

1. GitHub → repository → **Settings** → **Secrets and variables** → **Actions** → **New repository secret**.
2. Add:

Secret	Value
`AZURE_DEVOPS_REPO_URL`	the HTTPS clone URL from 4.1 (without `user@`)
`AZURE_DEVOPS_PAT`	the token from 5.1
`AZURE_ARTIFACTS_PAT`	a token with **Packaging: Read & write** (only if not already set in Part 2)

5.3 Test the sync

1. Make any small commit on GitHub (for example, edit the README on a feature branch) and push it.
2. GitHub → **Actions** → *Mirror to Azure DevOps* should be green within a minute.
3. Azure DevOps → **Repos** → **Commits**: the same commit appears with the same SHA.

> **Never commit directly in Azure Repos.** The mirror overwrites it on the next GitHub push and deletes anything that exists only there. All work goes through GitHub pull requests.

Alternative (if the PAT route is blocked): create the pipeline straight from GitHub. In 6.1 choose **GitHub** instead of *Azure Repos Git* and install the Azure Pipelines GitHub app. The pipeline then triggers from GitHub directly and the mirror is only a backup copy.

6. Create the pipeline

6.1 First run, build and test only

1. **Pipelines → New pipeline → Azure Repos Git** → `GiftOfTheGivers-HumanitarianAid` → **Existing Azure Pipelines YAML file** → branch `main`, path `/azure-pipelines.yml` → **Continue** → **Run**.
2. Rename it `GiftOfTheGivers-CI-CD` (⋮ → **Rename/move**).
3. If the run asks to **authorise** the `giftgivers-helpers` feed or other resources, click **Permit**.
4. With `deployEnabled: 'false'` (the default) only the **Build and test** stage runs. Check that:
   - the **Tests** tab shows three runs: *Unit tests*, *Integration tests*, *Selenium UI tests*, all passing;
   - the **Code Coverage** tab shows the merged report, and the *Coverage gate* step prints at least 70%;
   - the **Artifacts** list has `drop`, `migrations` and `ui-screenshots`.

If restore fails with *401 Unauthorized* for `giftgivers-helpers`: **Artifacts → giftgivers-helpers → ⚙ Feed settings → Permissions → Add users/groups** → add `<Project> Build Service (ST10438928)` as **Feed and Upstream Reader (Collaborator)**.

6.2 Turn on deployment

1. On GitHub, open `pipelines/templates/variables.yml` and set:
   - `deployEnabled: 'true'`
   - `webAppName`, `webAppUrl`, `resourceGroup`, `functionAppName`, `sqlServerName`, `sqlDatabaseName` to the names you wrote down;
   - `useStagingSlot: 'true'` and `webAppStagingUrl` **only** if you created the staging slot in 3.3;
   - `webAppType: webApp` and `functionAppType: functionApp` for Windows (the defaults).
2. Commit through a pull request into `main`. When it merges, the mirror copies it and the pipeline starts.
3. The run pauses at **DeployDatabase** waiting for approval (4.3). Open it → **Review → Approve**.
4. Watch the stages: *DeployDatabase* → *DeployWebApp* + *DeployFunctions* → *LiveSmokeTests*.

The first *DeployFunctions* run fails at **Test the deployed function**, because the function key is still `pending`. That is expected. Do section 8, then re-run that stage.

7. What each deployment stage does

Stage	Steps	Evidence for the report
Build and test	restore (locked) → build → unit → integration → Selenium (Chrome, desktop + mobile) → publish TRX → publish coverage → coverage gate ≥ 70% → vulnerability scan → EF model-drift check → `migrate.sql`, `rollback.sql`, `schema-version.txt` → zip web app and function	Tests tab, Code Coverage tab, artifacts
DeployDatabase	approval → show target schema version → run idempotent `migrate.sql` on Azure SQL → list `__EFMigrationsHistory`	the two SQL task logs (applied migrations are listed)
DeployWebApp	apply app settings + connection string → deploy zip (to `staging` if enabled) → smoke test → swap → smoke test production; **on failure: swap back**	deployment log, smoke-test HTTP codes
DeployFunctions	deploy zip → apply settings (connection string from the secret group) → call the function three ways: no key (401), bad reference (400), unknown reference (404)	the three HTTP results in the log
LiveSmokeTests	Selenium tests tagged *Smoke* against the live URL	Tests tab (*Live smoke tests*), `live-screenshots` artifact

8. After the first deployment

8.1 Function key

1. Portal → Function App `gift-relief-functions` → **Functions** → **GenerateTaxCertificate** → **Function Keys** → copy `default`.
2. Azure DevOps → **Library** → **giftgivers-azure** → set `functionKey` to that value (keep it secret) → **Save**.
3. Open the last run → **DeployFunctions** → **Rerun failed jobs**. It should now pass.
4. Manual test in a browser (replace the key): `https://gift-relief-functions.azurewebsites.net/api/tax-certificate/SEED000000000001?code=<key>` downloads the demo certificate PDF while demo data is on.

8.2 Check the live site

1. Open the web app URL. You should see the intro animation and then the home page.
2. Visit `/health`. It should say `Healthy` (this proves the database connection works).
3. Log in with the demo employee `employee@giftofthegivers.org` / `Employee#123` (present because `seedDemoData: 'true'`).

8.3 First real staff account

Before real users arrive:

1. Set `seedDemoData: 'false'` in `variables.yml` (removes demo seeding on new databases).
2. Register the real staff member on the live site.
3. App Service → **Environment variables** → add `StaffAccess__BootstrapEmployeeEmails__0` = that email → **Apply** (the app restarts and grants the Employee role).
4. Delete the demo accounts from **Staff Access** in the app.

9. Rollback and restore

What broke	How to roll back	Time
Web app, with a staging slot	Automatic: if a smoke test fails after the swap, the pipeline swaps back. Manually: portal → App Service → **Deployment slots** → **Swap** (staging ↔ production). The previous build is always in `staging` after a swap.	~1 min
Web app, no slot	Pipelines → **Runs** → open the last good run → **DeployWebApp** → **Rerun stage**. It redeploys that run's own `webapp.zip`.	~3 min
Function	Re-run **DeployFunctions** from the last good run.	~2 min
Database schema (latest migration is bad)	**Run pipeline** → tick **Roll the database back by one migration** → Run. The DB stage then runs `rollback.sql` (the `Down()` of the newest migration) instead of `migrate.sql`, and web/function deployment is skipped. Then revert the commit on GitHub.	~3 min
Database data (rows deleted or corrupted)	Portal → SQL database → **Restore** → *Point-in-time* → pick a time before the incident → restore as a new database (e.g. `GiftOfTheGiversDb-restored`) → check it → update `sqlConnectionString` in the variable group → re-run DeployWebApp and DeployFunctions. Basic tier keeps 7 days of backups.	15–30 min

The rollback and re-apply of the newest migration was rehearsed on SQL Server on 2026-11-02 (see the Part 3 report, section 6.3), and the CI workflow repeats it on every push.

10. Troubleshooting

Symptom	Cause and fix
Mirror workflow: `AZURE_DEVOPS_PAT is not configured`	Add the GitHub secrets (5.2). The names are case-sensitive.
Mirror workflow: `Authentication failed` / 401	PAT expired, revoked, wrong organisation or missing *Code: Read & write*. Create a new one.
Pipeline: *variable group giftgivers-azure could not be found*	The name is spelled differently, or the pipeline is not authorised: Library → group → **Pipeline permissions**.
Pipeline: *The pipeline is not valid … service connection*	Service connection name must equal `azureServiceConnection` in `variables.yml`, and it must be authorised for the pipeline.
Restore 401 on `giftgivers-helpers`	Give the Build Service identity *Feed Reader* (6.1).
SQL task: *Cannot open server … requested by the login* or firewall error	Turn on *Allow Azure services* on the SQL server (3.2, step 7). The task also adds a temporary rule for the agent IP. The service connection needs Contributor on the resource group.
Site shows **HTTP Error 500.30**	The app failed to start. App Service → **Log stream**. Usually a wrong connection string (check the *Connection strings* tab) or the database not migrated (re-run DeployDatabase).
Site still shows the Azure placeholder page	The DeployWebApp stage did not run (check `deployEnabled`, branch `main`, approval) or it deployed to `staging` without swapping.
`/health` returns `Unhealthy`	The database is unreachable: firewall, password, or the Basic database is paused (serverless).
Function smoke test 401 on every call	`functionKey` is still `pending` or wrong (section 8.1).
Function returns 500	Function App → **Monitor** / Application Insights. Usually `ConnectionStrings__DefaultConnection` is missing: re-run DeployFunctions.
Live Selenium smoke tests fail but the site works	The site was still warming up. Re-run the stage.

11. Hand-over checklist

Tick each item and keep the values in the team password manager.

✓	Item	Value / where
☐	Old shared PAT revoked	Azure DevOps → Personal access tokens
☐	Resource group	`rg-giftofthegivers`
☐	SQL server / database / admin login	`gift-relief-sql` / `GiftOfTheGiversDb` / password manager
☐	Web app name, URL, plan tier	portal Overview
☐	Staging slot created (S1+ only)	yes / no
☐	Function app name	`gift-relief-functions`
☐	Azure Repos clone URL	GitHub secret `AZURE_DEVOPS_REPO_URL`

✓	Item	Value / where
☐	Mirror PAT (Code R/W), expiry date	GitHub secret `AZURE_DEVOPS_PAT`
☐	Service connection	`giftgivers-azure-connection`
☐	Environment + approver	`giftgivers-production`
☐	Variable group with 4 secrets	`giftgivers-azure`
☐	Pipeline created and renamed	`GiftOfTheGivers-CI-CD`
☐	`deployEnabled: 'true'` merged to `main`	commit SHA
☐	Function key stored	`functionKey`
☐	Screenshots collected for the PoE	see the Part 3 report, section 10

12. Costs and clean-up

- Azure SQL Basic costs about US$5 a month. A Consumption function is free at this volume. The App Service cost depends on the existing plan (Free F1 = free; B1 ≈ US$13 a month; S1, needed for slots, ≈ US$70 a month, so use it only for the demonstration).
- After marking, delete the resource group to stop all charges: portal → `rg-giftofthegivers` → **Delete resource group**. Revoke the PATs as well.